

SISTEMA DE GESTIÓN “EDIFICIO MIRADOR”

(DAS) Documento Arquitectura de Software
Versión 1.0

Integrantes: Javiera Cid
Marjorie Portiño
Milton Saavedra



Identificación de Documento

Identificación	Informe de proyecto de software
Proyecto	Sistema de Gestión Edificio Mirador
Versión	1.0
Documento mantenido por	Equipo de desarrollo
Fecha de última revisión	03/04/2026
Fecha de próxima revisión	04/04/2026
Documento aprobado por	Profesor guía
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
02/04/2026	1.0	Creación inicio del documento.	Marjorie Portiño
03/04/2026	1.0	Se añadieron definiciones, acrónimos y abreviaciones.	Marjorie Portiño
04/04/2026	1.0	Inclusión de resumen ejecutivo y visión del sistema.	Marjorie Portiño
06/04/2026	1.0	Se agregaron los requisitos funcionales.	Javiera Cid
08/04/2026	1.0	Se agregaron los requisitos no funcionales y se realizó una revisión general del informe y últimas correcciones.	Javiera Cid
08/04/2026	1.0	Se agregó la arquitectura del sistema.	Milton Saavedra
15/05/2026	1.1	Se definió propósito de modelo 4+1, vista de escenario	Marjorie Portiño
16/05/2026	1.1	Se añadieron las descripciones generales de los cursos normal y alternativos.	Marjorie Portiño

18/05/2026	1.1	Se redactó la descripción del diagrama de clases y se incluyó en la Vista Lógica.	Marjorie Portiño
18/05/2026	1.1	Se incorporaron los diagramas de clases y casos de uso al documento.	Marjorie Portiño
18/05/2026	1.1	Se incorporó el diagrama de paquete y su descripción.	Javiera Cid
18/05/2	1.1	Se definió el propósito de la Vista de Procesos.	Javiera Cid
18/05/2026	1.1	Se incorporó el diagrama de actividad y su descripción.	Javiera Cid
20/05/2026	1.1	Incorporó diagrama de despliegue y descripción.	Milton Saavedra
20/02/2026	1.1	Incorporó diagrama de despliegue y descripción	Milton Saavedra
20/02/2026	1.1	Se agrego los requisitos de calidad	Milton Saavedra

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

INTRODUCCIÓN

1.1. Contexto del Problema

El edificio El Mirador, ubicado en una zona céntrica de la ciudad, enfrenta una serie de dificultades en la gestión en sus gastos comunes y en la administración de la comunidad. Con 160 departamentos y una alta rotación entre propietarios y arrendatarios, la administración debe coordinar múltiples intereses y necesidades.

Actualmente los procesos de cobro y registro de pagos se realizan de manera manual, lo que genera ineficiencias, errores y retrasos en la gestión.

La morosidad de los residentes afecta directamente el flujo de caja, provocando problemas de liquidez y retrasos en mantenciones esenciales. Además, la falta de un sistema automatizado dificulta la identificación de residentes morosos y reduce la transparencia en la comunicación entre la administración y la comunidad. Esta situación ha generado desconfianza, tensiones y aumento de los costos operativos, poniendo en riesgo la estabilidad financiera y la calidad de vida de los residentes.

1.2. Propósito

El propósito de este documento es definir los requisitos para el desarrollo del Sistema de Gestión de gastos comunes del edificio El Mirador, con el fin de optimizar los procesos administrativos y mejorar la transparencia en la relación con los residentes. Está dirigido a la administración del edificio, los residentes (propietarios y arrendatarios) y el equipo de desarrollo de software, con el objetivo de establecer el alcance del sistema y garantizar que cumpla con las necesidades del negocio y de los usuarios.

1.3. Ámbito

El sistema estará enfocado en mejorar la gestión de los gastos comunes del edificio El Mirador, apoyando a la administración en tareas como el registro de residentes, control de pagos, envío de notificaciones, la identificación de deudas y la generación de información clara para todos los involucrados. Además, permitirá a los residentes acceder fácilmente a sus datos, revisar su estado de pago y mantenerse informados. Este sistema será utilizado principalmente por el administrador del edificio y los residentes (propietarios y residentes), con el objetivo de mejorar la comunicación y la transparencia dentro de la comunidad.

Se centrará en la gestión interna del edificio y no incluirá el desarrollo de plataformas

externas de pago, aunque sí podrá adaptarse a futuras integraciones.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>SGC</i>	Sistema de Gestión de Gastos Comunes
<i>ER</i>	Especificación de requisitos
<i>Adm.</i>	Administrador del edificio
<i>UI</i>	Interfaz de Usuario
<i>Rep. Fin.</i>	Reportes Financiero
<i>WbApp</i>	Aplicación Web
<i>Not.</i>	Notificaciones

1.5. Resumen ejecutivo

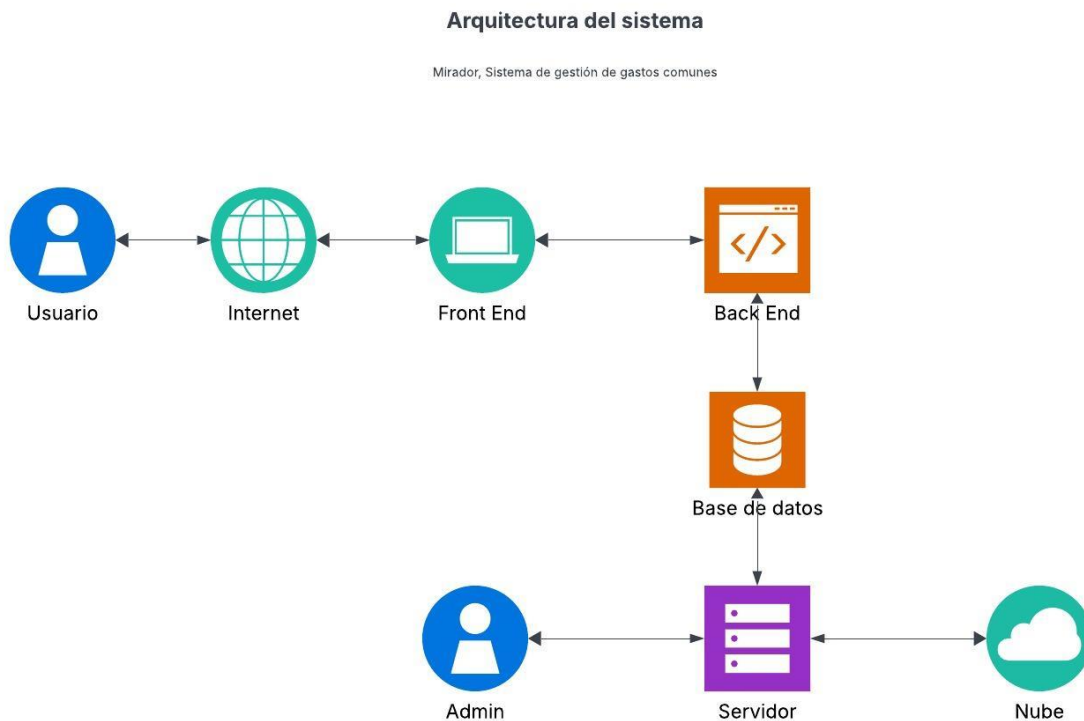
El Sistema de gestión de gastos comunes del edificio El Mirador, busca optimizar los procesos administrativos, reducir la morosidad y mejorar la transparencia en la información.

Actualmente, la administración enfrenta problemas debido a procesos manuales, falta de control en los pagos y dificultades en la comunicación con los residentes. Esto genera retrasos en mantenciones, conflictos y una gestión poco eficiente.

La solución propuesta consiste en un sistema automatizado que permita gestionar residentes, registrar pagos, emitir notificaciones y generar reportes, facilitando la toma

de decisiones y mejorando la convivencia dentro de la comunidad.

1.6. Arquitectura del sistema



2. VISIÓN DEL SISTEMA

Descripción general del sistema

El sistema será una plataforma web y móvil que permitirá a la administración y los residentes gestionar de manera eficiente los pagos, consultas y reportes asociados a la vida comunitaria del edificio El Mirador.

A través de esta solución, se podrán automatizar procesos como el registro de pagos, Control de morosidades y generación de informes, lo que ayudará a reducir errores y mejorar los tiempos de gestión.

Su propósito es optimizar los procesos administrativos, reducir la morosidad y

mejorar la transparencia en la comunicación entre la administración y los residentes, contribuyendo a una gestión más ordenada y eficiente.

2.1. Objetivos del sistema

Desarrollar un sistema que permita gestionar de manera eficiente, transparente y accesible los gastos comunes del edificio El Mirador.

- Optimizar la gestión administrativa de los gastos comunes
- Automatizar el registro y control de pagos.
- Reducir la morosidad mediante notificaciones y reportes.
- Facilitar el pago de cuotas y servicios asociados.
- Mejorar la organización y acceso a la información en tiempo real.
- Aumentar la satisfacción de los residentes mediante una gestión confiable y accesible.
- Fortalecer la transparencia y la comunicación entre la administración y los residentes.
- Disminuir los costos operativos asociados a procesos manuales.

2.2. Requerimientos funcionales.

Gestión de usuarios, roles y acceso

RF-01: Registro, actualización y desactivación de usuarios

RF-02: Asignación y actualización de roles y permisos

RF-03: Gestión de perfiles con control de acceso

Gestión de residentes y unidades

RF-04: Registro, actualización y desactivación de residentes

RF-05: Registro y actualización de departamentos

Gestión financiera y cobranza

RF-06: Cálculo automático de cuotas

RF-07: Registro y actualización de pagos de gastos comunes

RF-08: Registro de gastos extraordinarios

RF-09: Generación de recibos

RF-10: Gestión automática de morosidad

Gestión de notificaciones y comunicaciones

RF-11: Configuración de notificaciones

RF-12: Programación de notificaciones

RF-13: Notificaciones múltiples

RF-14: Historial de comunicaciones

Gestión de solicitudes y atención

RF-15: Registro y cierre de solicitudes de mantenimiento

RF-16: Registro y cierre de quejas

Consultas, visualización y control

RF-17: Acceso a información en tiempo real

RF-18: Consulta de pagos y deudas por residentes

RF-19: Consultas administrativas avanzadas

RF-20: Búsqueda y filtros

RF-21: Panel de control administrativo

Reportes y análisis

RF-22: Generación de informes generales

RF-23: Generación de reportes financieros

RF-24: Reporte de morosidad

RF-25: Reporte de pagos realizados

RF-26: Reporte de gastos comunes y extraordinarios

RF-27: Reporte de estado de cuenta por residente

RF-28: Reporte de solicitudes y mantenimiento

RF-29: Exportación de reportes

RF-30: Programación de reportes automáticos

2.3 Requerimientos no funcionales

1. Usabilidad y experiencia de usuario

RNF-01: Sistema intuitivo y fácil de usar

RNF-02: Accesibilidad visual y de navegación

RNF-03: Compatibilidad con navegadores y móviles

2. Rendimiento y capacidad operativa

RNF-04: Respuesta en ≤ 3 segundos

RNF-05: Soporte multiusuario

RNF-06: Escalabilidad

3. Disponibilidad y continuidad operativa

RNF-07: Disponibilidad $\geq 99\%$

RNF-08: Confiabilidad de datos

RNF-09: Respaldos automáticos y recuperación .

4. Seguridad y protección de la información

RNF-10: Autenticación segura

RNF-11: Protección de datos personales y financieros

RNF-12: Trazabilidad de operaciones críticas

5. Calidad de datos y confiabilidad

RNF-13: Validación de datos

RNF-14: Notificaciones confiables con reintentos

6. Mantenibilidad y evolución del sistema

RNF-15: Diseño modular para mantenimiento

RNF-16: Configurabilidad de reglas de negocio

RNF-17: Adaptación a idioma, moneda y normativas

2.3. Supuestos y dependencias

Supuestos

- Los residentes cuentan con acceso a internet.
- La administración utilizará el sistema de forma continua.
- Los usuarios poseen conocimientos básicos de tecnología.
- La información ingresada será correcta y actualizada.

Dependencias

- Disponibilidad de internet para el uso del sistema.
- Infraestructura tecnológica (servidor o hosting).
- Posible integración con métodos de pagos externos.
- Capacitación inicial para los usuarios.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

El sistema se diseñará con una arquitectura en capas, que se divide en tres niveles: presentación, lógica de negocio y datos. La capa de presentación corresponde a la interfaz web y móvil que usarán los residentes y el administrador. La capa de lógica de negocio se encargará de las reglas del sistema, como el cálculo de cuotas, la validación de pagos y la gestión de solicitudes. La capa de datos se ocupará de almacenar la información en la base de datos de manera segura.

3.2. Justificación del estilo según el contexto del sistema

Se eligió la arquitectura en capas porque el sistema es una aplicación web/móvil enfocada en la gestión interna del edificio. Este estilo permite separar responsabilidades, lo que hace que el sistema sea más fácil de mantener y más ordenado. Además, ayuda a reducir errores y facilita que se puedan agregar nuevas funciones sin afectar lo que ya está funcionando. Un sistema monolítico sería difícil de mantener y poco flexible, mientras que los microservicios serían demasiado complejos para este caso. Por eso, la arquitectura en capas es la más adecuada para el contexto del edificio El

Mirador.

3.3. Patrones de diseño aplicados

En este sistema se aplica principalmente el patrón MVC, porque ayuda a organizar mejor el programa y separar la lógica de negocio de la interfaz. Además, se usan patrones como Repositorio para manejar la base de datos, Singleton para asegurar que ciertos componentes tengan una sola instancia, y Observer para que las notificaciones funcionen automáticamente. Estos patrones hacen que el sistema sea más ordenado, fácil de mantener y que pueda crecer en el futuro sin tantos problemas.

3.4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El propósito de esta vista es mostrar cómo los diferentes actores del sistema interactúan con las funciones principales del Sistema de Gestión de Gastos Comunes del edificio el Mirador, Sirve para entender de manera general que hace cada actor y que espera obtener el sistema, sin entrar todavía en los detalles técnicos.

4.1.2. Actores

Administrador: actor principal del sistema. Gestiona usuarios, residentes, unidades, pagos, reportes y consultas.

Propietario: accede al sistema para revisar su estado de cuenta, realizar pagos, recibir notificaciones, realizar solicitudes y consultar información de su departamento.

Arrendatario: accede al sistema para revisar su estado de cuenta, realizar pagos, recibir notificaciones, realizar solicitudes y consultar información de su departamento.

Personal del edificio: participa en la gestión de solicitudes y atención, especialmente en mantenimiento y apoyo operativo.

Comité: colabora con el administrador en la revisión de reportes y análisis de gastos comunes.

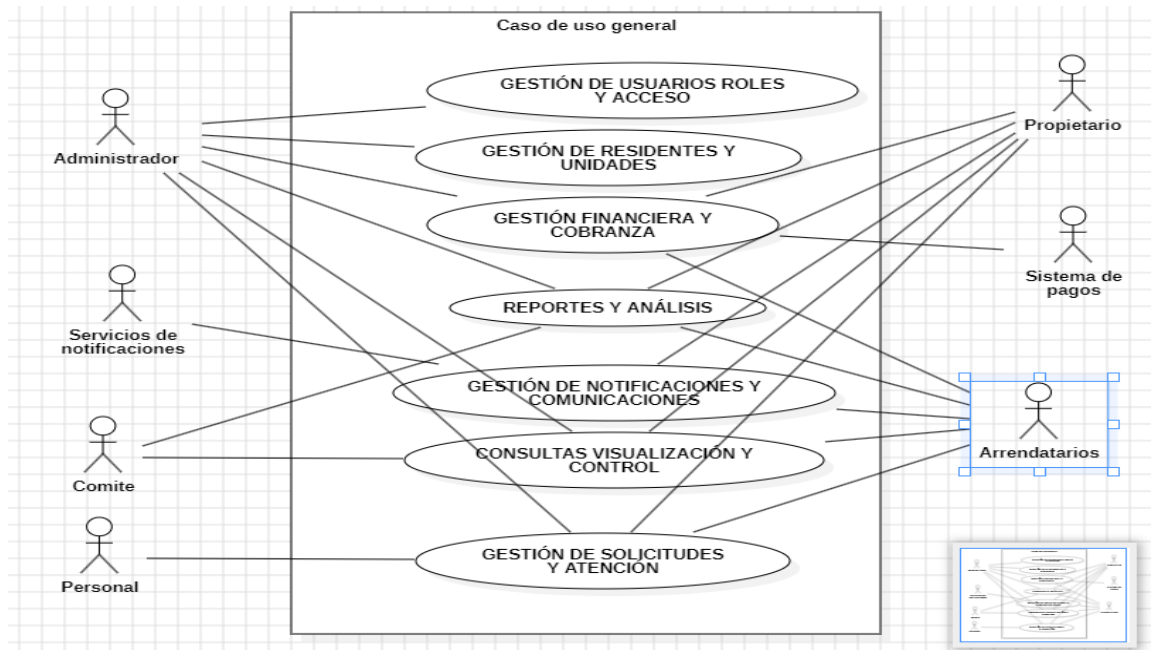
Sistema de pagos: actor externo que procesa las transacciones de los

residentes.

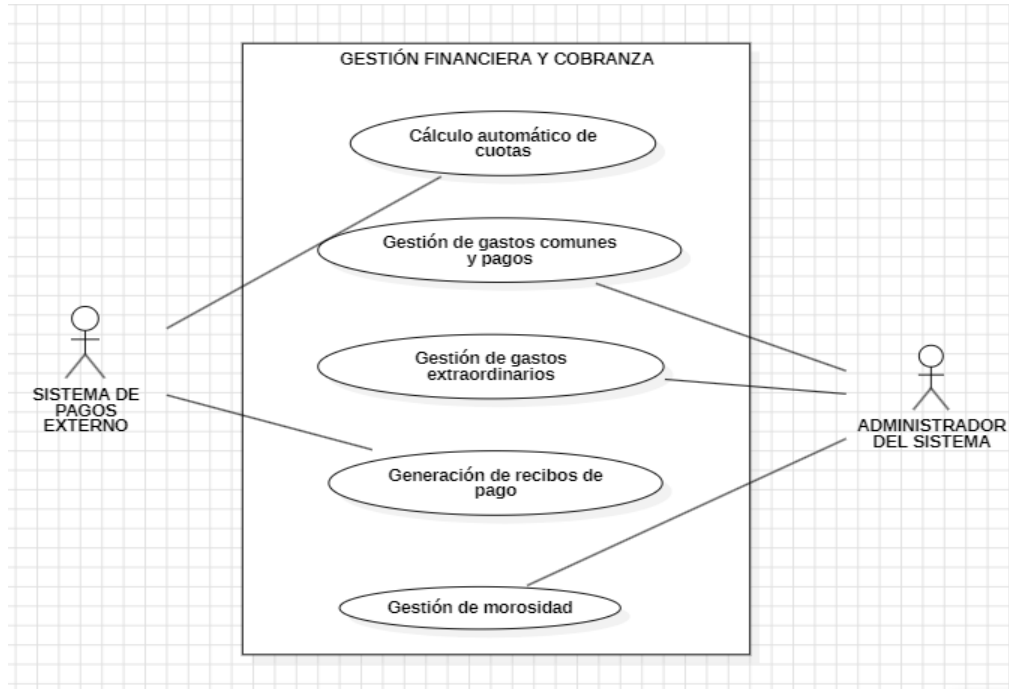
Servicio de notificaciones: actor externo encargado de enviar mensajes y

alertas automáticas a los residentes y al administrador.

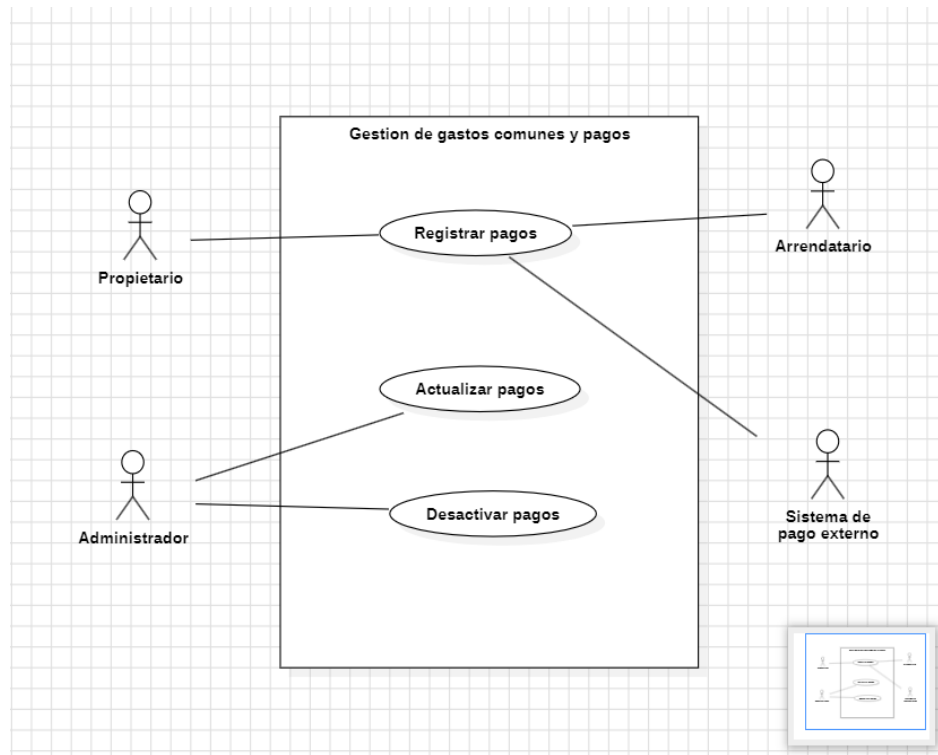
4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama intermedio de caso de uso



4.1.5. Diagrama de casos de uso específicos



4.1.6. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	Administrador, Sistema
CU-002-001	Exportar actividades	Administrador y sistema
CU-002-002	Importar deuda vencida por PDV	Administrador y sistema de pagos externo
CU-004-001	Generación Archivo PDA Importación	Administrador y sistema
CU-005-001	Registrar residente	Administrador, Residente (propietario/arrendatario)
CU-005-002	Consultar estado de cuenta	Residente, Sistema
CU-005-003	Realizar pago	Residente, Sistema de pagos externo
CU-006-001	Generar reporte de gastos comunes	Administrador, Comité, Sistema
CU-006-002	Revisar reporte	Comité, Administrador
CU-007-001	Registrar solicitud de mantenimiento	Residente, Personal del edificio
CU-007-002	Atender solicitud	Personal del edificio, Administrador
CU-008-001	Enviar notificación automática	Servicio de notificaciones, Residente
CU-008-002	Recibir notificación	Residente, Administrador
CU-009-001	Identificar morosidad	Sistema automatizado, Administrador
CU-009-002	Emitir recibo	Sistema, Administrador, Residente
CU-010-001	Gestionar usuarios	Administrador, Sistema
CU-010-002	Asignar roles	Administrador, Sistema
CU-011-001	Validar pagos	Sistema, Administrador, Sistema de pagos externo

4.1.7. Especificación de casos de uso

Caso de Uso	Registrar residente	Identificador: CU-005-001
Actores	Administrador, Residente	
Tipo	Primario	
Referencias	Gestión de residentes y unidades	
Precondición	El administrador debe haber iniciado sesión y tener permisos para registrar nuevos residentes	
Postcondición	El administrador debe haber iniciado sesión y tener permisos para registrar nuevos residentes.	
Descripción	Permite al administrador ingresar los datos del residente (nombre, rut, dirección, unidad, contacto) y guardarlos en el sistema	
Resumen	El administrador accede al módulo de residentes, completa los datos y el sistema guarda la información creando el registro del nuevo residente.	

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Ingresa al sistema con sus credenciales.
2	Sistema	Verifica los permisos del usuario.
3	Administrador	Selecciona la opción "Registrar residente".
4	Sistema	Muestra el formulario de registro.
5	Administrador	Completa los datos del residente y confirma
6	Sistema	Guarda la información y genera el registro.
7	Administrador	Visualiza el mensaje de confirmación.
El proceso inicia cuando el administrador accede al sistema y selecciona la opción para registrar un nuevo residente. El sistema valida los permisos, muestra el formulario y guarda la información ingresada. Finalmente, se genera el registro y se muestra un mensaje de confirmación.		

CURSO NORMAL

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2A	Si el administrador no tiene permisos, el sistema muestra un mensaje de acceso denegado y no permite continuar con el registro.
5A	Si el administrador deja campos obligatorios vacíos, el sistema muestra un mensaje de error y solicita completar la información antes de guardar.
6A	Si ocurre un error al guardar los datos, el sistema muestra un mensaje indicando que el registro no se pudo completar y ofrece la opción de reintentar.
En este flujo, pueden presentarse errores relacionados con permisos insuficientes, datos incompletos o fallos al guardar la información. El sistema responde mostrando mensajes de alerta y permite corregir los datos o reintentar la operación, garantizando la integridad del registro.	

Caso de Uso	Realizar pago	Identificador: CU-005-003
Actores	Residente, Sistema de pagos externo, Sistema	
Tipo	Primario	
Referencias	Gestión financiera y cobranza	
Precondición	El residente debe estar registrado y tener una deuda pendiente.	
Postcondición	El pago queda registrado y el saldo actualizado.	
Descripción	Permite que el residente realice el pago de sus gastos comunes mediante conexión con el sistema de pagos externo.	
Resumen	El residente selecciona su deuda, realiza el pago y el sistema actualiza automáticamente el estado de cuenta.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	Ingresa al sistema con sus credenciales.
2	Sistema	Verifica los datos y muestra el estado de cuenta.
3	Residente	Selecciona "Realizar pago".
4	Sistema	Envía la información al sistema de pagos externo.
5	Sistema de pago externo	Procesa la transacción y devuelve confirmación.
6	Sistema	Registra el pago y genera el comprobante
7	Residente	Visualiza el comprobante y finaliza la operación.
El proceso comienza cuando el residente accede al sistema y consulta su estado de cuenta. Luego selecciona la opción para realizar el pago, y el sistema envía la información al servicio de pagos externo. Una vez procesada la transacción, el sistema registra el pago, actualiza el saldo y genera el comprobante que el residente puede visualizar.		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2A	Si el residente ingresa credenciales incorrectas, el sistema muestra un aviso de error y no permite continuar con el proceso.
3A	Si el residente intenta realizar el pago sin tener deuda pendiente, el sistema muestra un mensaje indicando que no hay saldo por pagar.
4A	Si el sistema de pagos externo no responde, el sistema muestra un mensaje de error y solicita intentar nuevamente más tarde.
5A	Si el sistema de pagos externo rechaza la transacción (por fondos insuficientes o tarjeta inválida), se muestra un mensaje de rechazo y el residente debe ingresar otro medio de pago.
6A	Si ocurre un error al registrar el pago en la base de datos, el sistema muestra un mensaje de fallo y ofrece la opción de reintentar.
En este caso de uso pueden pasar cosas distintas a lo normal. Por ejemplo, si el residente escribe mal sus credenciales, el sistema no lo deja entrar. También puede pasar que no tenga deuda y entonces no se pueda pagar. A veces el sistema externo falla o rechaza la transacción, y el sistema muestra mensajes de error para que el residente intente otra vez o use otro medio de pago. Finalmente, si el sistema no logra guardar el registro, se avisa del fallo y se da la opción de reintentar.	

Caso de Uso	Generar reporte de gastos comunes	Identificador: CU-006-001
Actores	Administrador, Comité, Sistema	
Tipo	Primario	
Referencias	Gestión de reportes.	
Precondición	Deben existir datos de gastos registrados.	
Postcondición	Se genera reporte disponible para la revisión..	
Descripción	Permite generar reporte detallado de gastos comunes.	
Resumen	El administrador solicita el reporte y el sistema lo genera.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Administrador	Accede al módulo de reportes.
2	Sistema	Muestra opciones de generación.
3	Administrador	Selecciona periodo y confirma.
4	Sistema	Procesa datos.
5	Sistema	Genera reporte en PDF/Excel.
6	Administrador	Visualiza y guarda reporte.
7	Comité	Recibe reporte para revisión.
El administrador solicita el reporte, el sistema procesa los datos y genera el archivo, que luego es revisado por el comité.		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2A	No hay datos → mensaje “no existen gastos”.
4A	Error en procesamiento → mensaje de fallo
5A	Formato incorrecto → opción de reintentar.
En este caso de uso pueden aparecer situaciones distintas al flujo normal. Por ejemplo, si no existen datos de gastos, el sistema avisa que no hay información disponible. También puede ocurrir un error al procesar los datos, en cuyo caso se muestra un mensaje de fallo. Finalmente, si el reporte no se genera en el formato correcto, el sistema ofrece la opción de reintentar. De esta manera, el sistema siempre informa al administrador de los problemas y permite corregir o volver a ejecutar la acción.	

Caso de Uso	Registrar solicitud de mantenimiento	Identificador: CU-007-001
Actores	Residente, Administrador, Personal, Sistema	
Tipo	Primario	
Referencias	Gestión de solicitud	
Precondición	El residente debe estar autenticado	
Postcondición	La solicitud queda registrada y asignada	
Descripción	Permite registrar solicitudes de mantenimiento	
Resumen	El residente ingresa solicitud y el sistema la asigna	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	Ingresa al sistema.
2	Sistema	Valida credenciales.
3	Residente	Selecciona “Solicitud de mantenimiento”.
4	Sistema	Muestra formulario
5	Residente	Completa datos y confirma.
6	Sistema	Guarda solicitud.
7	Administrador	Asigna al personal.
8	Personal	Recibe notificación
El residente registra la solicitud, el sistema la guarda y el administrador la asigna al personal, quien recibe notificación.		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2A	Credenciales inválidas → acceso denegado .
5A	Datos incompletos → mensaje de error.
6A	Error al guardar → opción de reintentar.
5A	Error al guardar → opción de reintentar
7A	Sin personal disponible → mensaje de espera
En este caso de uso pueden ocurrir situaciones distintas al flujo normal. Por ejemplo, si el residente escribe mal sus credenciales, el sistema no lo deja entrar. También puede pasar que falten datos en el formulario y el sistema pida completarlos. A veces el sistema falla al guardar la solicitud y muestra un mensaje para intentar otra vez. Finalmente, si no hay personal disponible, la solicitud queda en espera hasta que se pueda asignar	

Caso de Uso	Enviar notificación automática	Identificador: CU-008-001
Actores	Sistema, Servicio de notificaciones, Residente	
Tipo	Secundario	
Referencias	Comunicación y alertas.	
Precondición	Debe existir evento que dispare la notificación.	
Postcondición	El residente recibe la notificación.	
Descripción	El sistema envía notificación automática al residente.	
Resumen	El sistema detecta evento y envía notificación.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Sistema	Detecta evento (ej. deuda vencida).
2	Sistema	Verifica los datos y muestra el estado de cuenta.
3	Sistema	Prepara el mensaje.
4	Sistema	Envía datos al servicio externo.
5	Sistema de pago externo	Procesa la transacción y devuelve confirmación.
6	Servicio externo	Entrega mensaje al residente.
7	Residente	Recibe y visualiza notificación.
El residente registra la solicitud, el sistema la guarda y el administrador la asigna al personal, quien recibe notificación.		

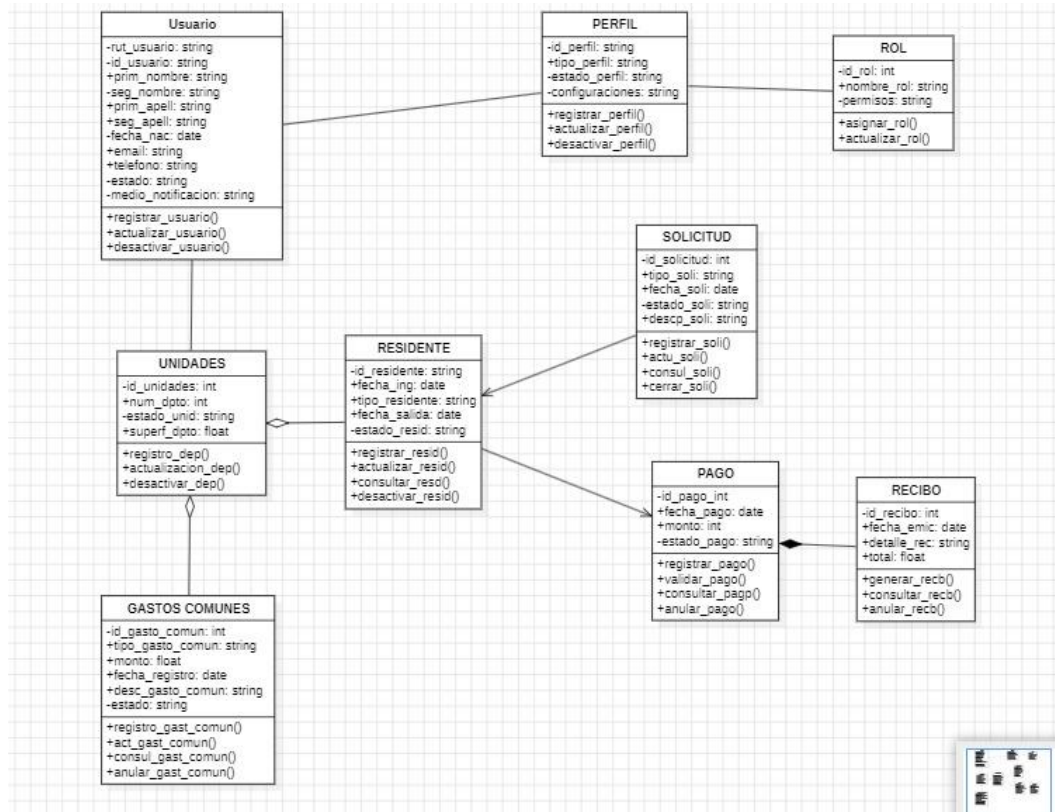
Nro.	Descripción de acciones alternas
2A	Credenciales inválidas → acceso denegado
5A	Datos incompletos → mensaje de error
6A	Error al guardar → opción de reintentar.
7A	Sin personal disponible → mensaje de espera .
En este caso de uso pueden pasar situaciones distintas al curso normal. Por ejemplo, si el residente escribe mal sus credenciales, el sistema no lo deja entrar. También puede ocurrir que falten datos en el formulario, que falle el guardado o que no haya personal disponible. En todos esos casos, el sistema muestra mensajes de error o espera y da la opción de corregir o intentar de nuevo.	

4.2 VISTA LÓGICA

- 4.1.8. El propósito de esta vista es mostrar la estructura lógica del sistema El Mirador, representando las clases principales y sus relaciones. Permite entender cómo se organiza la información y cómo interactúan los distintos

componentes del sistema para cumplir las funciones del proyecto.

4.1.9. Diagrama de clases



4.1.10. Descripción diagrama de clases

El diagrama de clases representa la estructura lógica del sistema El Mirador. Se muestran las clases principales como Usuario, Perfil, Rol, Residente, Unidad, Solicitud, Pago, Recibo, Notificación y Gastos Comunes, junto con sus atributos y métodos.

Las líneas entre las clases indican las relaciones que permiten la interacción entre ellas. Por ejemplo, el Usuario se conecta con Perfil y Rol, el Residente se relaciona con Unidad, y las clases financieras (Pago, Recibo, Morosidad, Gastos Comunes) muestran cómo se gestionan las transacciones.

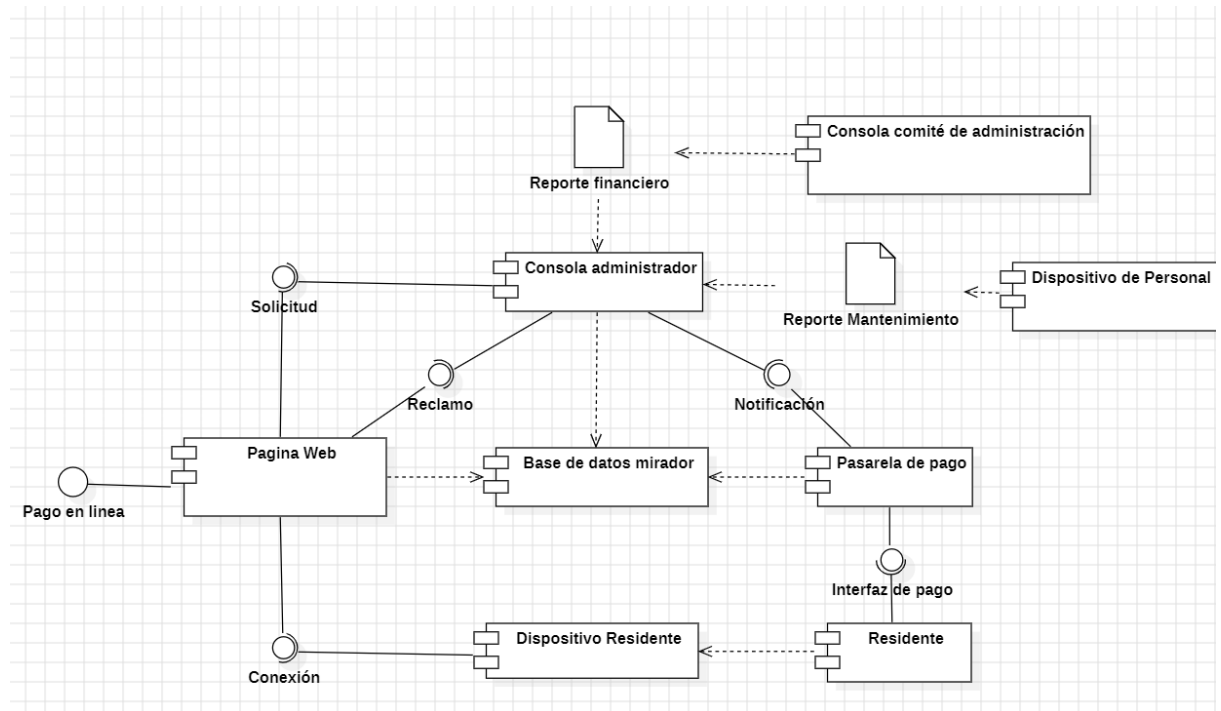
4.2. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.2.1. Propósito

La función de la vista de Implementación y Desarrollo es organizar todos los componentes del software otorgando un entendimiento de cada

segmento del software y cómo interactúan entre sí.

4.2.2. Diagrama de componente

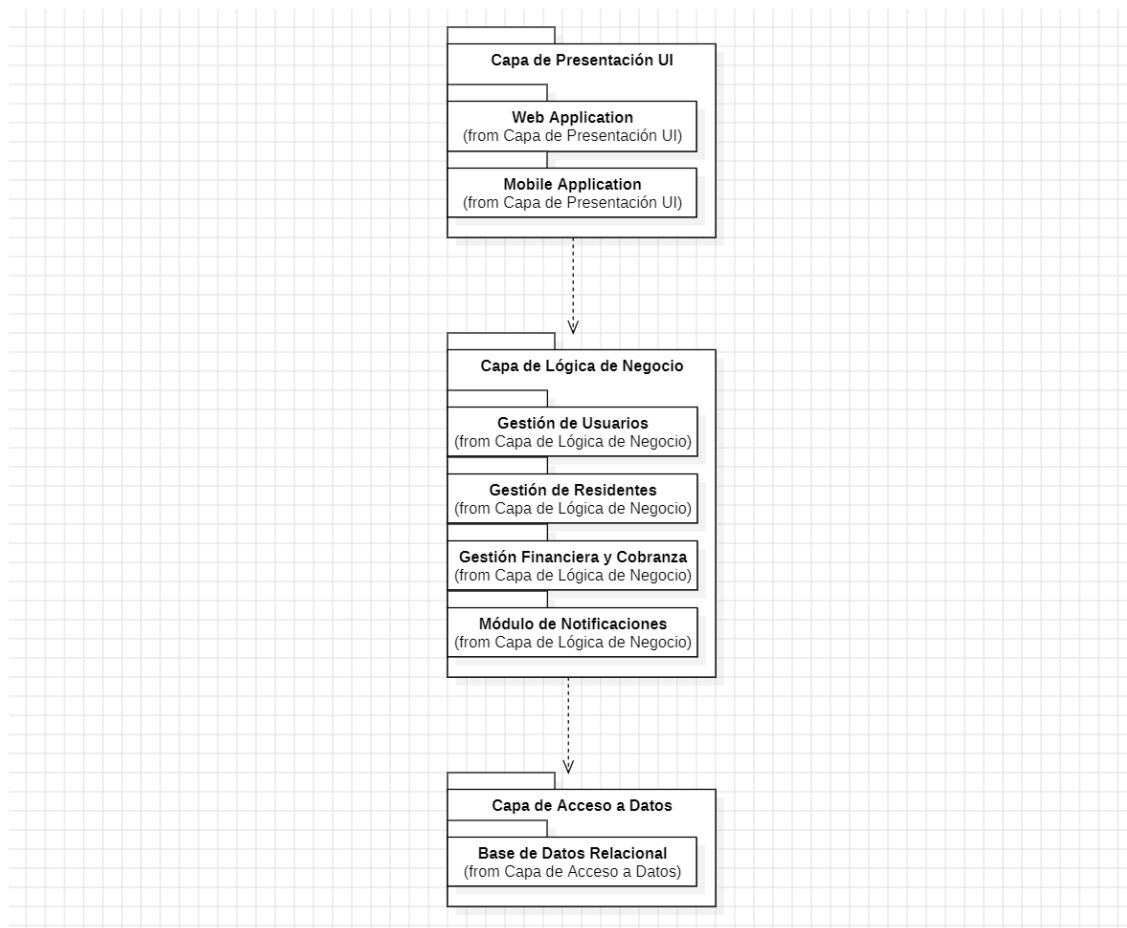


4.2.3. Descripción diagrama de componente

El diagrama de componentes muestra cómo se conectan distintos fragmentos físicos del sistema, esto se hace dividiendo las partes del software en módulos y mostrar donde se necesitan y conectan distintas interfaces que se crean cuando se comunican entre sí estos módulos. Este diagrama sirve para mostrar cómo funciona el software de manera

general.

4.2.4. Diagrama de paquete



4.2.5. Descripción diagrama de paquete

El Diagrama de Paquetes organiza el código fuente del sistema del Edificio "El Mirador" bajo un patrón de Arquitectura en Tres Capas. Su objetivo principal es asegurar la modularidad del software, dividiendo el trabajo de desarrollo y reduciendo la dependencia directa entre la interfaz visual, las reglas lógicas y el almacenamiento de datos. Esto facilita un mantenimiento sencillo y permite que el sistema crezca de manera escalable sin generar conflictos en el código.

El diagrama muestra una jerarquía ordenada de forma vertical que se comunica de

arriba hacia abajo mediante relaciones de dependencia lógica (flechas discontinuas de tipo «use» y «access»):

1. Capa de Presentación UI (Superior): Aloja los canales visuales de interacción para los usuarios, distribuidos en dos subpaquetes de front-end: Web Application (panel para el administrador) y Mobile Application (acceso rápido para residentes y conserjes).
2. Capa de Lógica de Negocio (Central): Es el motor de las reglas comunitarias y operativas del edificio. Contiene cuatro subpaquetes de procesamiento interno:
 - Gestión de Usuarios: Control de accesos y perfiles de seguridad.
 - Gestión de Residentes: Registro de departamentos y copropietarios.
 - Finanzas y Cobranza: Cálculo automatizado de gastos comunes, deudas y cobros.
 - Módulo de Notificaciones: Despacho automático de avisos por correo electrónico o alertas.
3. Capa de Acceso a Datos (Inferior): Encargada de la persistencia de la información. Protege el subpaquete de la Base de Datos Relacional (MySQL), donde se almacenan físicamente todos los registros del sistema.

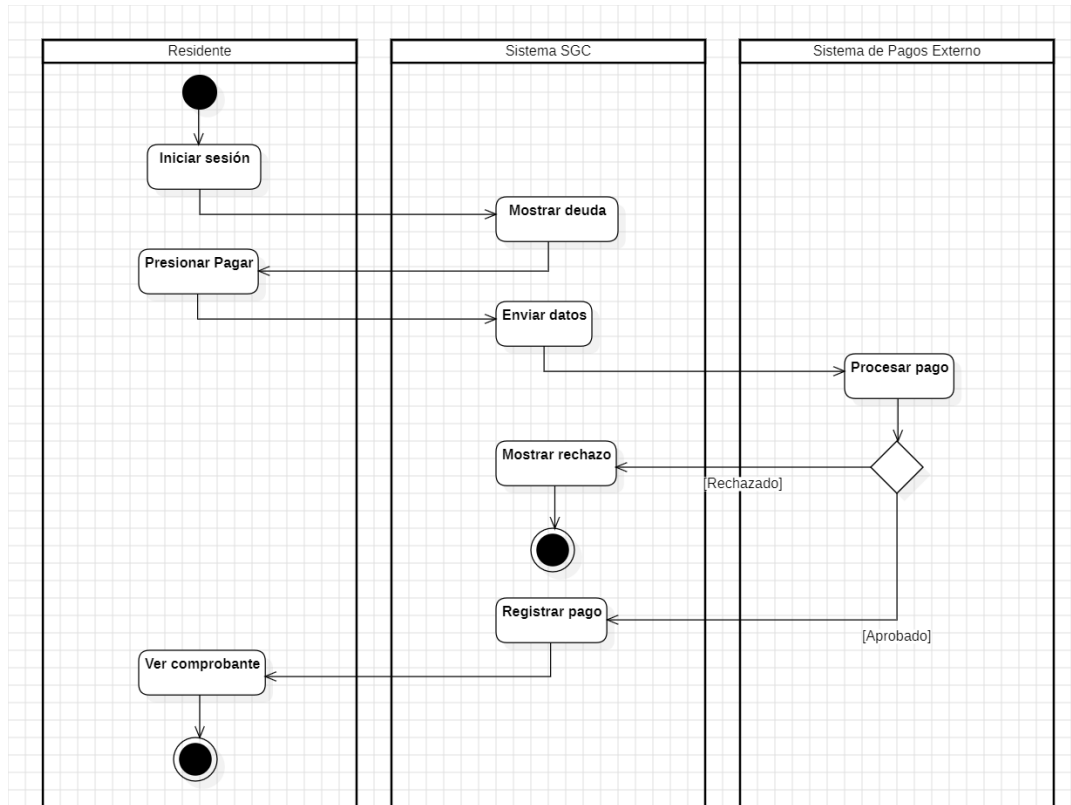
4.3. VISTA DE PROCESOS

4.3.1. Propósito

La Vista de Procesos en el modelo 4+1 se enfoca en el comportamiento dinámico del sistema en tiempo de ejecución. Su propósito es describir los flujos de trabajo, la secuencia de pasos lógicos y la interacción entre los diferentes actores y componentes del software. Para el edificio "El Mirador", esta vista permite validar la integridad y la seguridad de las transacciones financieras y de comunicación más

críticas de la comunidad, asegurando un control sin errores.

4.3.2. Diagrama de actividad (Específico)



4.3.3. Descripción diagrama de actividad

Este diagrama modela el flujo operativo del caso de uso crítico "Realizar Pago de Gastos Comunes". Sirve para representar de manera clara cómo interactúan el usuario residente, el software interno y el banco externo. Su objetivo es asegurar que el sistema controle correctamente tanto el pago aprobado, actualización de saldo y emisión de comprobante como los escenarios de error, fallas en la transacción y mensajes de rechazo, garantizando la transparencia de las cuentas de la comunidad.

El diagrama visualiza una secuencia en zigzag estructurada en tres carriles de responsabilidad (Swimlanes) que dividen el proceso de forma lógica:

1. Carril del Residente (Actor Inicial): Controla las acciones en la interfaz. Muestra desde el Inicio, el inicio de sesión, la confirmación de pago al presionar el botón correspondiente y la visualización final del comprobante digital.
2. Carril del Sistema SGC (Software Central): Ejecuta la lógica interna del edificio. Muestra la deuda consultada en base de datos, el empaquetado seguro de la información para despacharla al banco, y el resultado final: registra el pago y actualiza saldos si es exitoso, o bien gatilla la acción de mostrar un rechazo y cierra con un Fin con Error si falla.
3. Carril de la Pasarela Externa (Banco / Transbank): Procesa la transacción real. Contiene la acción de cobro bancario y un Rombo de Decisión

condicional que evalúa la operación:

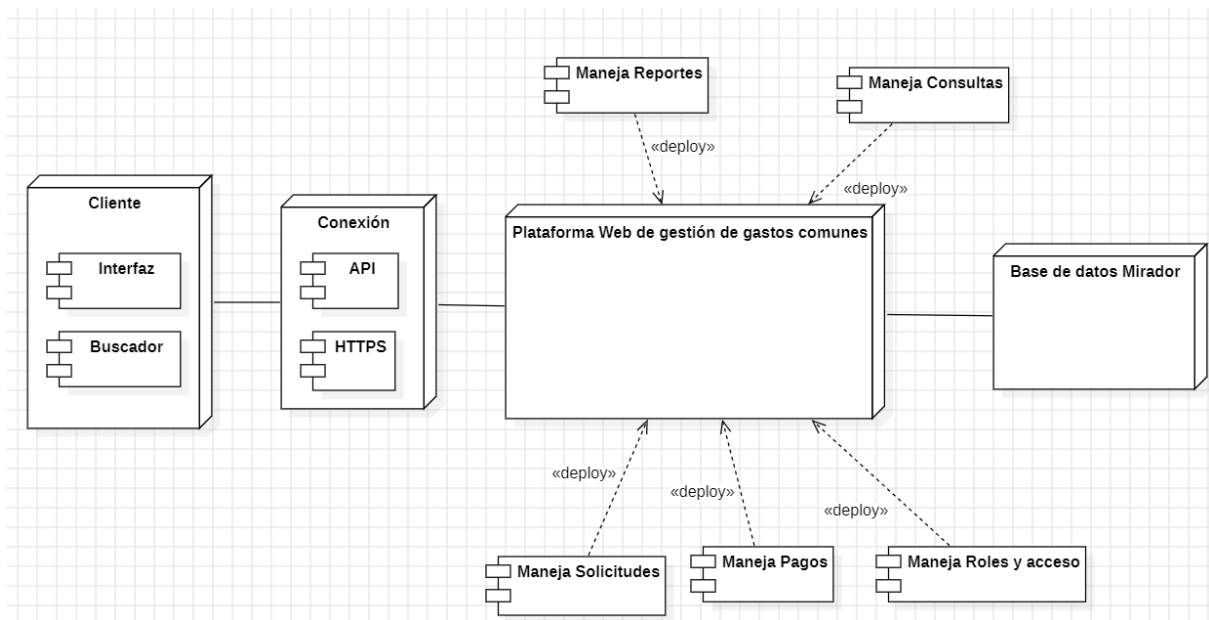
- Ruta [Aprobado]: Redirige el flujo al Sistema SGC para consolidar la transacción con éxito y finalizar en el Fin Exitoso en el carril del Residente.
- Ruta [Rechazado]: Redirige el flujo al carril central para alertar al usuario del error.

4.4. VISTA FÍSICA

4.4.1. Propósito

El propósito de la vista física se usa para mostrar el software como debería quedar y funcionar al ser desplegado.

4.4.2. Diagrama de despliegue



4.4.3. Descripción diagrama de despliegue

Muestra físicamente cómo será la integración del software, esto se hace con nodos y artefactos representando como se conectan una parte a otra del software.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito de esta sección es definir los criterios de calidad que debe

cumplir el sistema El mirador para asegurar su correcto funcionamiento, confiabilidad y satisfacción de los usuarios. Estos requisitos permiten evaluar si el software cumple con los estándares esperados en cuanto a rendimiento, seguridad, usabilidad y mantenibilidad.

5.2. Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN
Usabilidad	Facilidad con la que los usuarios pueden aprender y usar el sistema.	El sistema será utilizado por residentes y administradores con conocimientos básicos de tecnología, por lo que debe ser intuitivo.
Accesibilidad (WCAG)	Cumplimiento de estándares de accesibilidad web (WCAG nivel AA).	Permite que personas con distintas capacidades puedan usar el sistema sin barreras.
Rendimiento	Tiempo de respuesta y capacidad de procesar múltiples solicitudes.	El sistema debe responder rápido para evitar frustración y mejorar la eficiencia.
Mantenibilidad	Facilidad para actualizar y corregir el sistema sin afectar otras funciones.	El sistema debe evolucionar con nuevas necesidades sin generar errores.
Seguridad	Protección de datos personales y financieros mediante autenticación y trazabilidad	La información sensible de residentes y pagos debe estar protegida.
Portabilidad	Capacidad de funcionar en distintos dispositivos y navegadores.	Los residentes accederán desde móviles y computadores, por lo que debe ser compatible.

5.3. Reglas y criterios de evaluación de calidad

Reglas de calidad

- Tiempo de carga en operaciones críticas ≤ 3 segundos.
- Cumplimiento de accesibilidad según WCAG nivel AA.
- Disponibilidad $\geq 99\%$ anual.
- Autenticación segura y trazabilidad de operaciones críticas.
- Modularidad en el diseño para facilitar el mantenimiento.
- Compatibilidad con navegadores principales y dispositivos móviles.

Criterios de evaluación de calidad

Usabilidad: Pruebas con usuarios y puntuación Nielsen de heurísticas.

Accesibilidad: Validación con herramientas automáticas (ej. WAVE) y revisión

manual de WCAG AA.

Rendimiento: Pruebas de carga midiendo tiempo de respuesta (< 3 seg).

Mantenibilidad: Inspecciones de código y pruebas de regresión tras actualizaciones.

Seguridad: Pruebas de penetración y validación de autenticación segura.

Portabilidad: Pruebas en distintos navegadores (Chrome, Firefox, Edge) y dispositivos (PC, móvil).

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt

CARTA GANTT PROYECTO SOFTWARE																	
Metodología de Gestión: Cascada. Las fases se ejecutan de manera secuencial, una despues de la otra.																	
FASE	ENTREGABLE	ROLES DEL PROYECTO	DURACION Hr-Dia	COSTO HOR	COSTO TOTAL	\$1	\$2	\$3	\$4	\$5	\$6	\$7	\$8	\$9			
PLANIFICACIÓN	Cronograma inicial	Jefe de proyecto	16	14400	230000												
PLANIFICACIÓN	Estimación de recursos	Analista	24	9200	220800												
PLANIFICACIÓN	Costos	Jefe de proyecto	20	14400	288000												
PLANIFICACIÓN	Plan de riesgos	Jefe de proyecto	20	14400	288000												
ANALISIS	Levantamiento de información	Analista	40	9200	368000												
ANALISIS	Estudio de factibilidad	Jefe de proyecto	40	14400	576000												
ANALISIS	Documento de alcance	Analista	40	9200	368000												
REQUISITOS	Especificación de requisitos	Analista	80	9200	736000												
REQUISITOS	Casos de uso	Analista	40	9200	368000												
REQUISITOS	Diagramas de actores	Ingeniero software	40	14000	560000												
REQUISITOS	Documento SRS	Mixto	80	30000	1800000												
DISEÑO	Arquitectura de sistemas	Jefe de proyecto	60	14400	864000												
DISEÑO	Modelo entidad relacion	Ingeniero software	40	14000	560000												
DISEÑO	Diagramas UML	Ingeniero software	60	14000	840000												
DISEÑO	Mockup de sistema	Analista	40	9200	368000												
DESARROLLO	Implementación logica de negocio	Ingeniero software	120	25000	3000000												
DESARROLLO	Integración con APIs	Ingeniero software	40	14000	560000												
DESARROLLO	Configuración de entornos	DevOps	24	12800	307200												
DESARROLLO	Control de versiones (Git)	Ingeniero software	16	14000	224000												
DESARROLLO	Documentación tecnica	Ingeniero software	32	14000	448000												
PRUEBAS	Pruebas unitarias	QA Tester	40	8600	344000												
PRUEBAS	Pruebas de integración	QA Tester	40	8600	344000												
PRUEBAS	Pruebas de sistemas	QA Tester	40	8600	344000												
PRUEBAS	Pruebas de integración	QA Tester	40	8600	344000												
PRUEBAS	pruebas de aceptación del usuario	QA Tester	40	8600	344000												
DESPLIEGUE	Configuración de servidores	DevOps	40	12800	512000												
DESPLIEGUE	Procesos de Integración y Despliegue	DevOps	40	12800	512000												
DESPLIEGUE	Migración de datos	Analista	40	9200	368000												
DESPLIEGUE	Capacitación de datos	Capacitador TI	40	11500	460000												
MANTENIMIENTO	Reparar errores reportados	Ingeniero software	40	14000	560000												
MANTENIMIENTO	Ajustar software a nuevos entornos	Ingeniero software	40	14000	560000												
MANTENIMIENTO	Incorporar nuevas funciones	Ingeniero software	40	14000	560000												